

Zebra invalidateblock Remote Denial-of-Service Vulnerability

Summary

After receiving a specially crafted invalidateblock administrative request, a Zebra node can erase the only chain state it is using. Zebra provides three official default configurations; once the RPC configuration is used, see <https://github.com/ZcashFoundation/zebra/blob/v6.2.3/docker/docker-compose.lwd.yml>.

Any attacker can terminate the node with a single request, without an account or password. The same request can be sent again after the node restarts, causing repeated outages.

Mitigating Factors:

- Use either of the other two default configurations.
- Current evidence shows that the issue only stops the node. No evidence was found that it can be used to steal funds, read private data, or create a consensus fork.
- The crash log records a clear panic location and call stack, making it relatively easy for administrators to distinguish the incident from an ordinary network failure and identify the cause.

Aggravating Factors:

- The official Lightwalletd Docker example enables RPC and disables cookie authentication. If the port is exposed to the public Internet or an untrusted network, anyone can remotely shut down the node without a password. This turns a vulnerability normally protected by the default configuration into a directly exposed public attack surface.
- The attack cost is almost zero: it requires only one small RPC request, no computing power, no forged block, and no high-volume traffic.
- Even if an administrator restarts the node, the attacker can send the request again, turning a one-time crash into repeated outages or long-term unavailability.
- The interface may return "success" before the node crashes, so monitoring systems or callers may incorrectly assume that the operation completed normally, making diagnosis more difficult.
- After the node exits, it can no longer provide services. Without reliable automatic restart and network isolation, manual intervention is required, and wallets, indexers, or other dependent systems remain disconnected during the outage.

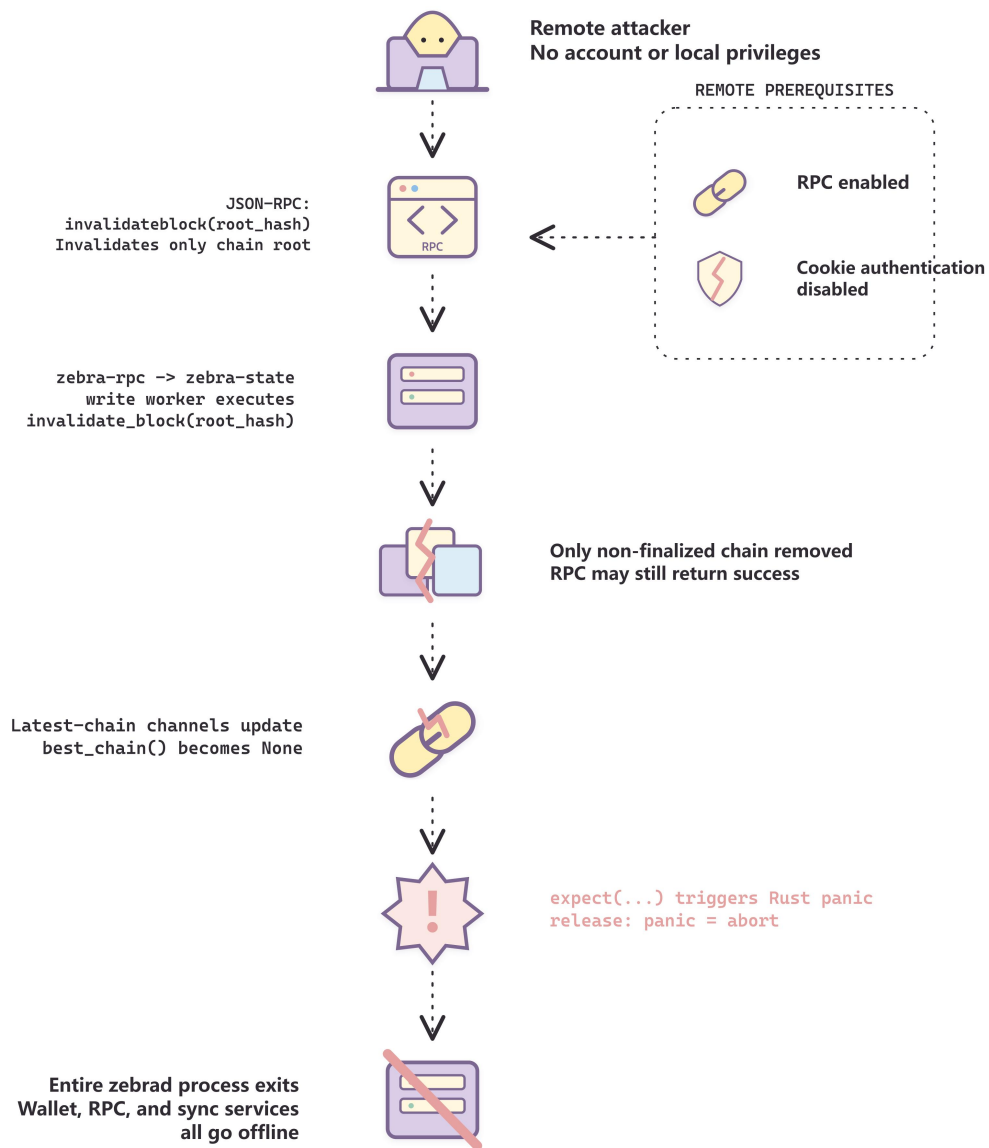
Vulnerability Details:

The core issue is that invalidateblock allows an external caller to delete the only chain in the non-finalized state, while other parts of the system still treat "the non-finalized state is never empty" as an unbreakable internal invariant. When an ordinary block is invalidated, the code preserves the chain segment before the target block, so chain_set still contains one chain. When the root block of the only chain is invalidated, however, there is no preceding segment to retain; chain_set becomes empty and best_chain() returns None.

The complete call chain is shown below:

RPC Remote Denial-of-Service Attack Chain

From one anonymous request to full zebra process termination



Technical Details:

Root Cause 1: Deletion of the Only Non-Finalized Chain Is Allowed (zebra-state/src/service/non_finalized_state.rs)

In `invalidate_block()`, when the target is the root block, the code removes the entire chain from `chain_set` without checking whether it is the last chain, whether `chain_set` will be empty afterward, or whether the subsequent chain-tip notification logic can represent the state "only finalized state exists; no non-finalized chain exists":

```

let invalidated_blocks = if chain.non_finalized_root_hash() == block_hash {
    self.chain_set.remove(&chain);
    chain.blocks.values().cloned().collect()
} else {
    // Standard block invalidation path
};

```

The deletion itself is not necessarily wrong. The actual problem is that other parts of the system continue to treat "the non-finalized state is never empty" as an unbreakable internal invariant.

Root Cause 2: Unconditional Best-Chain Update After the Operation (zebra-state/src/service/write.rs)

After the write worker processes an invalidation request, it unconditionally calls `update_latest_chain_channels()` for messages that have no new block to commit. It does not consider whether `invalidate_block()` succeeded, whether a best non-finalized chain still exists after the operation, or whether the finalized chain tip should be published instead:

```

NonFinalizedWriteMessage::Invalidate { hash, rsp_tx } => {
    let _ = rsp_tx.send(non_finalized_state.invalidate_block(hash));
    None
}
// ...
let Some((queued_child, rsp_tx)) = queued_child_and_rsp_tx else {
    update_latest_chain_channels(...);
    continue;
};

```

Another important timing detail is that `rsp_tx.send(...)` executes before the chain channels are updated. The client therefore receives a successful RPC response before the node panics.

Root Cause 3: `expect()` Is Used for an Invariant That External Input Can Break

Inside `update_latest_chain_channels()`:

```

let best_chain = non_finalized_state.best_chain().expect(
    "unexpected empty non-finalized state: must commit at least one block before
    updating channels"
);

```

This `expect()` was intended to express an internal invariant, but the `invalidateblock` RPC can directly violate that invariant through external input. The panic is therefore reachable from an external interface.

Why the Panic Terminates the Entire Node

The workspace release configuration uses `panic = "abort"`. Unlike `panic = "unwind"`, which attempts to unwind the current thread stack and could theoretically isolate the failure to a task or thread,

abort immediately terminates the entire process without stack unwinding. Consequently, even though the panic occurs in the state write worker, the entire zebra process exits.

Dynamic Evidence

Latest-version test environment: zebra v6.3.0 (release build, isolated Regtest). Key log output:

```
invalidating a block in the non-finalized state
hash=block::Hash("b1766552...")

thread '<unnamed>' panicked at zebra-state/src/service/write.rs:99:55:
unexpected empty non-finalized state: must commit at least one block before
updating channels

update_latest_chain_channels at write.rs:99:55
run at write.rs:374:17
```

Minimal Reproduction Procedure (isolated Regtest only; this operation terminates the node)

1. Create a test state containing only one non-finalized chain
2. Call `getblockhash(1)` to obtain the non-finalized root block hash
3. Call `invalidateblock(root_hash)`
4. The RPC returns `result: null`
5. The state write worker panics while updating the best chain
6. zebra exits because `panic=abort`

The decisive condition is not a fixed height of 1. The target must actually be the root block of the only current non-finalized chain.

Security Impact

- A single request can terminate the node process.
- The issue can be triggered again after the process restarts.
- The node must be restarted manually or by a service manager.
- No information disclosure, financial loss, or consensus fork has been demonstrated.
- The primary impact is complete loss of availability.

Long-Term Remediation Recommendations

- In `invalidate_block()`, reject requests that would delete the final non-finalized chain and return a clear RPC error, or formally support an empty non-finalized state.
- `update_latest_chain_channels()` must not call `expect()` directly on an empty state. When the state is empty, it should publish the finalized tip or skip the non-finalized channel update.
- Invoke the existing chain-channel update logic only when the invalidation succeeds and a best chain still exists.
- Add release-profile regression tests to ensure that invalidating the only root block cannot cause a panic.
- Retain Zebra's general default cookie authentication and do not expose RPC on untrusted networks. For official deployment examples that must disable cookie authentication for Lightwalletd compatibility, bind to loopback or a trusted network by default and block anonymous external access with a firewall, reverse-proxy authentication, or equivalent access controls.

Recommended regression tests:

- Test 1: Invalidate a normal non-root block -> succeeds and the node continues running.
- Test 2: Invalidate the root of the only chain -> returns a controlled error or safely transitions to an empty state.
- Test 3: Invalidate a nonexistent hash -> returns an error and the node continues running.
- Test 4: All tip/watch channels remain consistent after the operation.
- Test 5: Execute under the release panic policy and confirm that the process does not terminate.